

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

P2_WINDOW = ["0,0", "0,1", "1,2", "2,1", "1,0", "0,0"]
N_QUBITS = 3
D = 2 ** N_QUBITS
P_NOISE = 0.01

P2_ENCODING = {
    "0,0": "000",
    "0,1": "001",
    "1,0": "010",
    "1,2": "011",
    "2,1": "100",
    "3,3": "101",
}

def ket_from_bitstring(bitstring):
    index = int(bitstring, 2)
    v = np.zeros((D, 1), dtype=complex)
    v[index, 0] = 1.0
    return v

def pure_density(label):
    v = ket_from_bitstring(P2_ENCODING[label])
    return v @ v.conj().T

def von_neumann_entropy(rho, base=2):
    rho = (rho + rho.conj().T) / 2
    vals = np.linalg.eigvalsh(rho)
    vals = np.clip(vals.real, 0.0, None)
    vals = vals[vals > 1e-14]

    if len(vals) == 0:
        return 0.0

    return float(-np.sum(vals * np.log(vals)) / np.log(base))

def purity(rho):

```

```
return float(np.trace(rho @ rho).real)
```

```
def report_density(rho):  
    eigvals = np.linalg.eigvalsh((rho + rho.conj()).T / 2)  
    eigvals = np.clip(eigvals.real, 0.0, None)  
  
    return {  
        "trace": float(np.trace(rho).real),  
        "purity": purity(rho),  
        "S_vN_bits": abs(von_neumann_entropy(rho)),  
        "eigenvalues": np.round(eigvals, 12)  
    }
```

```
I1 = np.array([[1, 0], [0, 1]], dtype=complex)  
X1 = np.array([[0, 1], [1, 0]], dtype=complex)  
Z1 = np.array([[1, 0], [0, -1]], dtype=complex)
```

```
def expand_single_qubit_operator(op, target, n_qubits):  
    ops = []  
  
    for q in range(n_qubits):  
        if q == target:  
            ops.append(op)  
        else:  
            ops.append(I1)  
  
    full = ops[0]  
    for next_op in ops[1:]:  
        full = np.kron(full, next_op)  
  
    return full
```

```
def apply_kraus_channel(rho, kraus_ops):  
    out = np.zeros_like(rho, dtype=complex)  
  
    for K in kraus_ops:  
        out += K @ rho @ K.conj().T  
  
    return out
```

```

def depolarizing_channel(rho, p):
    return (1 - p) * rho + p * np.eye(D, dtype=complex) / D

def bit_flip_channel(rho, p, target_qubit=0):
    X = expand_single_qubit_operator(X1, target_qubit, N_QUBITS)
    return (1 - p) * rho + p * (X @ rho @ X.conj().T)

def phase_flip_channel(rho, p, target_qubit=0):
    Z = expand_single_qubit_operator(Z1, target_qubit, N_QUBITS)
    return (1 - p) * rho + p * (Z @ rho @ Z.conj().T)

def amplitude_damping_channel(rho, gamma, target_qubit=0):
    K0_1q = np.array([
        [1, 0],
        [0, np.sqrt(1 - gamma)]
    ], dtype=complex)

    K1_1q = np.array([
        [0, np.sqrt(gamma)],
        [0, 0]
    ], dtype=complex)

    K0 = expand_single_qubit_operator(K0_1q, target_qubit, N_QUBITS)
    K1 = expand_single_qubit_operator(K1_1q, target_qubit, N_QUBITS)

    return apply_kraus_channel(rho, [K0, K1])

def phase_damping_channel(rho, gamma, target_qubit=0):
    K0_1q = np.sqrt(1 - gamma) * I1

    K1_1q = np.sqrt(gamma) * np.array([
        [1, 0],
        [0, 0]
    ], dtype=complex)

    K2_1q = np.sqrt(gamma) * np.array([
        [0, 0],
        [0, 1]
    ], dtype=complex)

```

```

K0 = expand_single_qubit_operator(K0_1q, target_qubit, N_QUBITS)
K1 = expand_single_qubit_operator(K1_1q, target_qubit, N_QUBITS)
K2 = expand_single_qubit_operator(K2_1q, target_qubit, N_QUBITS)

```

```

return apply_kraus_channel(rho, [K0, K1, K2])

```

```

NOISE_CHANNELS = {
    "none": lambda rho: rho,
    "depolarizing_at_1_0": lambda rho: depolarizing_channel(rho, P_NOISE),
    "bit_flip_at_1_0": lambda rho: bit_flip_channel(rho, P_NOISE, target_qubit=0),
    "phase_flip_at_1_0": lambda rho: phase_flip_channel(rho, P_NOISE, target_qubit=0),
    "amplitude_damping_at_1_0": lambda rho: amplitude_damping_channel(rho, P_NOISE,
target_qubit=0),
    "phase_damping_at_1_0": lambda rho: phase_damping_channel(rho, P_NOISE,
target_qubit=0),
}

```

```

all_records = []
final_records = []

```

```

for channel_name, channel_fn in NOISE_CHANNELS.items():
    step_records = []

```

```

    for step, label in enumerate(P2_WINDOW):
        rho = pure_density(label)
        event = "P2 state"

```

```

        if label == "1,0":
            rho = channel_fn(rho)
            event = f"{channel_name}"

```

```

        if step == len(P2_WINDOW) - 1 and label == "0,0":
            rho = pure_density("0,0")
            event = "final 0,0 forced reset"

```

```

    metrics = report_density(rho)

```

```

    row = {
        "channel": channel_name,
        "step": step,
        "P2_state": label,
    }

```

```

        "event": event,
        "trace": metrics["trace"],
        "purity": metrics["purity"],
        "S_vN_bits": metrics["S_vN_bits"],
        "eigenvalues": metrics["eigenvalues"]
    }

    step_records.append(row)
    all_records.append(row)

    final_row = step_records[-1].copy()
    final_records.append(final_row)

df_all = pd.DataFrame(all_records)
df_final = pd.DataFrame(final_records)

print("\n===== FINAL P2 ROBUSTNESS OUTPUT =====")
print("Window: first 0,0 -> final 0,0")
print("Noise site: 1,0 only")
print(f"Noise probability / gamma: {P_NOISE}")
print(f"Hilbert dimension: {D}")

print("\nFinal forced 0,0 results by noise channel:")
print(
    df_final[
        [
            "channel",
            "P2_state",
            "trace",
            "purity",
            "S_vN_bits",
            "eigenvalues"
        ]
    ].to_string(index=False)
)

print("=====")

df_all.to_csv("p2_noise_sweep_all_steps.csv", index=False)
df_final.to_csv("p2_noise_sweep_final_outputs.csv", index=False)

plt.figure(figsize=(10, 5))

```

```

for channel_name in NOISE_CHANNELS.keys():
    subset = df_all[df_all["channel"] == channel_name]
    plt.plot(
        subset["step"],
        subset["S_vN_bits"],
        marker="o",
        label=channel_name
    )

plt.xticks(range(len(P2_WINDOW)), P2_WINDOW)
plt.xlabel("P2 node")
plt.ylabel("Von Neumann entropy S(rho), bits")
plt.title("P2 Entropy Robustness Sweep: Noise Localized at 1,0")
plt.legend()
plt.tight_layout()
plt.show()

```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

P2_WINDOW = ["0,0", "0,1", "1,2", "2,1", "1,0", "0,0"]
P_DEPOL = 0.01
N_QUBITS = 3
D = 2 ** N_QUBITS

P2_ENCODING = {
    "0,0": "000",
    "0,1": "001",
    "1,0": "010",
    "1,2": "011",
    "2,1": "100",
    "3,3": "101",
}

```

```

def ket_from_bitstring(bitstring):
    index = int(bitstring, 2)
    v = np.zeros((D, 1), dtype=complex)
    v[index, 0] = 1.0
    return v

def pure_density(label):
    v = ket_from_bitstring(P2_ENCODING[label])
    return v @ v.conj().T

def depolarize_at_1_0(rho, p):
    return (1 - p) * rho + p * np.eye(D, dtype=complex) / D

def von_neumann_entropy(rho, base=2):
    rho = (rho + rho.conj().T) / 2
    vals = np.linalg.eigvalsh(rho)
    vals = np.clip(vals.real, 0.0, None)
    vals = vals[vals > 1e-14]
    return float(-np.sum(vals * np.log(vals)) / np.log(base))

def purity(rho):
    return float(np.trace(rho @ rho).real)

records = []

for step, label in enumerate(P2_WINDOW):
    rho = pure_density(label)
    event = "P2 state"

    if label == "1,0":
        rho = depolarize_at_1_0(rho, P_DEPOL)
        event = "1,0 localized depolarization"

    if step == len(P2_WINDOW) - 1 and label == "0,0":
        rho = pure_density("0,0")
        event = "final 0,0 forced reset"

    eigvals = np.linalg.eigvalsh((rho + rho.conj().T) / 2)

```

```

eigvals = np.clip(eigvals.real, 0.0, None)

records.append({
    "step": step,
    "P2_state": label,
    "event": event,
    "trace": np.trace(rho).real,
    "purity": purity(rho),
    "S_vN_bits": von_neumann_entropy(rho),
    "eigenvalues": np.round(eigvals, 12)
})

df = pd.DataFrame(records)

print("P2 window: first 0,0 to final 0,0")
print(f'Depolarization only at 1,0, p = {P_DEPOL}')
print(f'Hilbert dimension d = {D}')
print(df.to_string(index=False))

final = df.iloc[-1]
print("\nFinal output:")
print(f'Final state: {final["P2_state"]}')
print(f'Final event: {final["event"]}')
print(f'Trace: {final["trace"]}')
print(f'Purity: {final["purity"]}')
print(f'Von Neumann entropy, bits: {abs(final["S_vN_bits"])}')
print(f'Eigenvalues: {final["eigenvalues"]}')

plt.figure(figsize=(9, 4.5))
plt.plot(df["step"], df["S_vN_bits"].abs(), marker="o")
plt.xticks(df["step"], df["P2_state"])
plt.xlabel("P2 node")
plt.ylabel("Von Neumann entropy S(rho), bits")
plt.title("P2 Entropy: First 0,0 to Final 0,0")
plt.tight_layout()
plt.show()

```